

Inclusions croisées

Exemple d'utilisation

Ce tutoriel présente un exemple de programme en C qui nécessite des inclusions croisées. Il pourrait servir de base pour représenter un document JSON en mémoire.

1 Présentation des structures

Le but de l'application est de représenter des éléments qui peuvent être des entiers, des objets ou des tableaux. Les valeurs des membres des objets sont eux-mêmes des éléments. Les tableaux sont des tableaux d'éléments.

Les éléments, définis dans le fichier d'en-tête `element.h`, sont représentés par une structure `element_t` qui contient un type (qui peut être soit `TYPE_INT`, `TYPE_ARRAY` ou `TYPE_OBJECT`) et une valeur. Celle-ci correspond à l'union `value_t` qui possède des membres de type `int`, `array_t` et `object_t`.

Au moment de la déclaration de l'union, les structures `array_t` et `object_t` ne sont pas connues. Il faut donc les déclarer avant de déclarer l'union `value_t` (nous utilisons ici `typedef` pour simplifier l'usage).

```
typedef struct array_t array_t;
typedef struct object_t object_t;

typedef union {
    struct array_t *array;
    struct object_t *object;
    int integer;
} value_t;
```

Le langage C permet cela à une condition : les champs doivent être des pointeurs. En effet, lors de la compilation, il est nécessaire de connaître la taille précise des structures/unions, ce qui n'est pas possible si toutes les membres n'ont pas une taille calculable.

Les déclarations des structures `array_t` et `object_t` sont déclarées de manière classique (respectivement dans les fichiers d'en-tête `array.h` et `object.h`). Notez cependant le nommage de la structure (le `array_t` avant les accolades) qui est obligatoire pour respecter la déclaration faite précédemment.

```
typedef struct array_t {
    int length;
    element_t **cells;
} array_t;

typedef struct {
    char name[MAX_STR];
    element_t *value;
} member_t;

typedef struct object_t {
    int nb;
    member_t *members;
} object_t;
```

2 Inclusions

Chaque fichier `.c` inclut son propre fichier d'en-tête. Dans le cas de `object.c`, si on inclut directement le fichier `object.h`, une erreur de compilation est affichée : la structure `element_t` n'est pas connue. Il suffit donc d'ajouter l'inclusion de `element_t` avant. Le fonctionnement est similaire pour `array.c`.

Pour `element.c`, c'est un peu différent. Comme dans le fichier d'en-tête `element.h` nous avons déclaré les structures, il n'est pas nécessaire (et surtout il ne faut pas le faire) d'inclure les fichiers d'en-tête `array.h` et `object.h` avant. Par contre, comme dans le code nous avons besoin des fonctions associées, il faut les inclure après.

Le programme de test donc le code est dans le fichier `example.c` crée les différents éléments possibles. Nous avons donc besoin d'inclure `element.h` puis `array.h` et `object.h`.